PREVIOUS NEXT

L1: Types of Programming Languages

Lesson Introduction

Hundreds of programming languages are currently used in the IT industry. Each one serves a specific purpose and has its own characteristics. In this lesson, you will learn how to classify languages based on the **programming paradigm** (object-oriented, procedural, or functional), **conversion type** (interpreted or compiled), and **typing style** (static or dynamic and strong or weak).

The estimated time: **25 minutes**

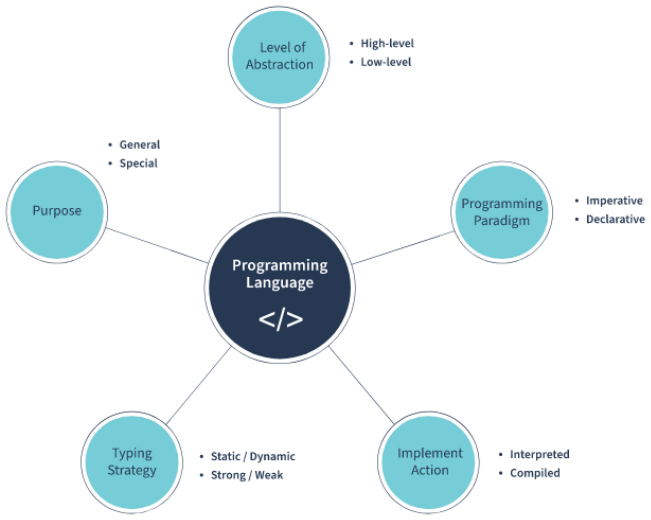
Types of Programming Languages

Python has been gaining popularity since its release in 1991 and is currently the most in-demand language in the world (TIOBE Index).

Apr 2022	Apr 2021	Change	Programming Language	Ratings	Change
1	3	↑	Python	13.92%	+2.88%
2	1	↓	C	12.71%	-1.61%
3	2	↓	Java	10.82%	-0.41%
4	4		C++	8.28%	+1.14%
5	5		C#	6.82%	+1.91%
6	6		Visual Basic	5.40%	+0.85%
7	7		JavaScript	2.41%	-0.03%
8	8		Assembly language	2.35%	+0.03%
9	10	↑	SQL	2.28%	+0.45%
10	9	↓	PHP	1.64%	-0.19%

But why is Python so popular? How is it different from other languages, and does it have any limitations? To answer these questions, take a look at the most common characteristics of programming languages.

Each language can be described using the following characteristics:



Now, explore each one in detail.

Purpose

Languages can differ in terms of the scope of tasks they perform.

General programming languages	Special programming languages
These languages can solve various tasks without specific language constructions. Examples: - Python - Rust - Java - Lisp	These languages were created to solve specific kinds of problems, which imposes certain restrictions on how they are used. Examples: - Maple for mathematical computations - SQL for database queries

Level of Abstraction

Another characteristic is the level of abstraction of the programming language when interacting with hardware.

Low-level languages	High-level languages
	
Low-level languages send direct calls to the OS and are represented in 0 or 1 forms. They have little to no abstraction. Low-level programs work fast and consume very little memory. **Examples of low-level languages:** - Machine code - Assembler	**High-level languages** send indirect calls to the OS using abstraction level. Such programs don't depend on hardware and require a **compiler** or **interpreter** to translate them to the OS. **Examples of high-level languages:** - Python - Java - Ruby - JavaScript

Programming Paradigm

Some languages are better suited for a specific way of programming than others.

There are two main programming paradigms: **declarative** and **imperative**. These terms don't refer to specific languages but to the programming styles they employ.

Click the arrows to navigate through the items below.

Declarative paradigm

Declarative languages describe the intended **result** of a program without specifying how to achieve it. You can use a language's features, extensions, or libraries to do this.

Suppose you want a pizza. According to the declarative paradigm, you describe what kind of pizza you want— pepperoni—but not how it is prepared. It can be ordered from a restaurant or cooked in a kitchen. The process doesn't matter to you.

Examples of declarative languages:

- HTML—You don't need to know how a browser renders HTML. You describe the structure of the webpage.
- SQL—You don't need to know how queries work. You just describe the result.

Both declarative and imperative paradigms can be subdivided into even more specific ways of programming:

Click each heading to see more information.

- **Functional paradigm**
- **Object-oriented paradigm**
- **Procedural paradigm**

Implementation

High-level languages make developing a program easier and faster than low-level languages. However, the OS doesn't directly understand source code written using some high-level language. The special programs, which are called compiler and interpreter, allow transforming source code to code understandable by the OS. Based on this, there are two large groups of languages:

Click each heading to see more information.

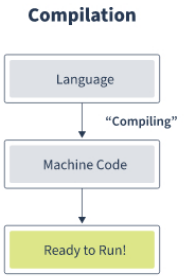
Compiled languages

Compilation is the process of translating source code from high-level programming language to lower-level language (e.g., assembly code, object code, or machine code) to create an executable program ("Compiler" (2022) Wikipedia).

Advantages:
- Faster execution than interpreted language

Disadvantages:
- It takes time to compile the program before its execution
- Compiled code depends on the compilation platform

Examples of compiled languages:
- C
- Go
- Pascal



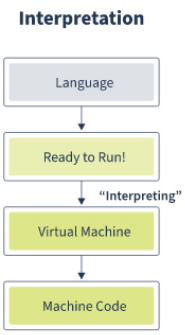
Interpreted languages

Interpretation transforms source code to bytecode (an intermediate representation language), which is then executed by the interpreter step by step. Interpreted languages are cross-platform but take longer to execute than compiled languages.

Advantages:
- Platform-independent code
- Smaller programs than compiled languages

Disadvantages:
- Slower execution

Examples of interpreted languages:
- Python
- Lisp
- PHP



0 points possible

Test Your Knowledge

*This is a **non-graded** quiz. The result of this quiz is **not** going to be included into your cumulative score for the course.*

Read each question below and select each correct answer from the dropdown menu. Then click "submit."

Which of the following features is a major benefit of **compiled** languages?

- ○ Platform-independence
- ◉ Fast execution
- ○ Small programs
- ○ Step-by-step interpretation

✔

Answer
Correct: Code compilation in compiled languages occurs before runtime, making programs run faster.

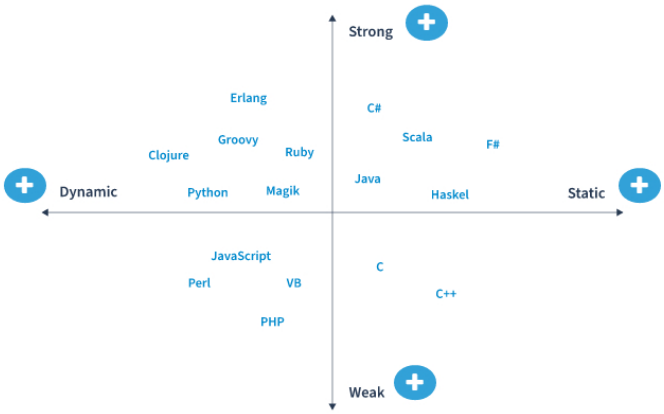
Submit | Reset | Show answer

Another method of classifying languages is by typing the strategy they implement.

Typing Strategy

Type-checking is the process of verifying the type of construct and its usage context. This helps to minimize the possibility of type errors in the program.

Click each + button in the graphic below to see more information.



Take a look at some examples of code:

Click the arrows to navigate through the items below.

Strong vs. Weak

Strong / Python

```
number = 42 + "666"
TypeError: Unsupported operand type(s)
for +: "int" and "str"

number = str(42) + "666"
"42666"

number = 42 + int("666")
708

number = "42" + "666"
"42666"
```

Weak / JavaScript

```
var number = 42 + "666"
"42666"
```

0 points possible

Test Your Knowledge

*This is a **non-graded** quiz. The result of this quiz is **not** going to be included into your cumulative score for the course.*

A programming language doesn't support automatic conversion of data types and checks code before execution.

Read the question below and select the correct answer. Then click "submit."

What kind of language is this describing?

- (●) Static - Strong
- ( ) Static - Weak
- ( ) Dynamic - Strong
- ( ) Dynamic - Weak

✔

Answer
Correct:
It is a static - strong language. In static languages, checks occur before runtime, which helps detect bugs quickly. Strong typing doesn't implicit data type based on the situation. So you have to declare each variable type.

Submit | Reset | Show answer

Lesson Summary

This lesson provided a survey of the various classifications of programming languages: abstraction level, programming style, interpretation, and printing method. In the next lesson, you will see Python's place among other programing languages and learn more about its history and current trends.

‹ PREVIOUS NEXT ›